

ScoobyDoo - IIT Bombay ICPC Team Notebook

(2019-20)

Contents

1	String	
1.1	Aho-corasick	
1.2	KMP	
1.3	Z-Algorithm	
1.4	Suffix Array	
1.5	Manacher's Algorithm	
1.6	Palindrome Tree	
2	Combinatorial optimization	
2.1	Dinic	
2.2	Max bipartite matching	
2.3	Min Cost Max Flow	
2.4	Hopcroft Karp	
3	Geometry	
3.1	2D Geometry	
4	Numerical algorithms	
4.1	Number theory	
4.2	Systems of linear equations, Determinant	
4.3	Fast Fourier transform	
4.4	NTT	
4.5	Online NTT	
4.6	Karatsuba	
5	Graph algorithms	
5.1	Strongly connected components and Topological Sort	
5.2	Eulerian path	
5.3	2 SAT	
5.4	SPFA	
5.5	Bridge Tree	
5.6	Heavy Light Decomposition	
5.7	Centroid Decomposition	
6	Data Structures	
6.1	Treap	
6.2	Persistent Segment Tree	
7	Important Formulae	
7.1	Partition Numbers	
7.2	Transformation matrix for XOR FFT	
7.3	Transformation matrix for AND FFT	
7.4	Transformation matrix for OR FFT	
8	Templates	
8.1	Inbuilt functions and Data Structures	

1 String

1.1 Aho-corasick

```
const int K = 26;
struct Vertex {
    int next[K];
    bool leaf = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch)
    {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].leaf = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
```

```

    if (t[v].next[c] != -1)
        t[v].go[c] = t[v].next[c];
    else
        t[v].go[c] = v == 0 ? 0 : go(get_link(
            v), ch);
}
return t[v].go[c];
}

```

1.2 KMP

```

const int MAXN = 1e5 + 5;
int f[MAXN]; // Failure function
string s, t; // s = pattern; t = text
void build() {
    f[0] = 0;
    int i = 1, j = 0;
    while(i < s.size()) {
        if(s[i] == s[j]) {
            f[i] = j+1;
            i++; j++;
        } else if(j == 0) {
            f[i] = 0; i++;
        } else {
            j = f[j-1];
        }
    }
}
void search(vector<int> &ans) {
    int i = 0, j = 0;
    while(i < t.size()) {
        if(j == s.size()) {
            ans.push_back(i-j);
            j = f[j-1];
        } else if(t[i] == s[j]) {
            i++; j++;
        } else if(j == 0) {
            i++;
        } else {
            j = f[j-1];
        }
    }
    if(j == s.size()) {
        ans.push_back(i-j);
    }
}

```

1.3 Z-Algorithm

```

// count number of occurrences of a in t
vector<int> z_function(string s) {
    int n = (int) s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r)
            z[i] = min (r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[
            i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}

```

1.4 Suffix Array

```

const int MAXN = 100005;
const int LOGN = 18;
struct block {
    int pos[2], ind;
};
bool cmp(block &a, block &b) {
    return (a.pos[0] == b.pos[0]) ? (a.pos[1] < b.
        pos[1]) : (a.pos[0] < b.pos[0]);
}
string a;
block blk[MAXN];
int pos[LOGN][MAXN], sufArr[MAXN], lcp[MAXN], lgn,
n;
int LCP(int u, int v) {
    if(u == v) return n - u;
    int ans = 0;
    for(int i = lgn - 1; i >= 0 && u < n && v < n;
        i--) {
        if(pos[i][u] == pos[i][v]) {
            u += (1<<i); v += (1<<i); ans += (1<<i);
        }
    }
    return ans;
}

```

```

}
void buildSA() {
    n = a.size();
    for(int i = 0; i < n; i++) {
        pos[0][i] = a[i] - 'a';
    }
    int add;
    for(lgn = 1, add = 1; add < 2*n; lgn++, add
        <= 1) {
        for(int i = 0; i < n; i++) {
            blk[i].pos[0] = pos[lgn-1][i];
            blk[i].pos[1] = (i+add < n)?pos[lgn-1][i+add]:-1;
            blk[i].ind = i;
        }
        sort(blk, blk + n, cmp);
        for(int i = 0; i < n; i++) {
            if(i > 0 && blk[i].pos[0] == blk[i-1].pos[0] && blk[i].pos[1] == blk[i-1].pos[1]) {
                pos[lgn][blk[i].ind] = pos[lgn][blk[i-1].ind];
            } else {
                pos[lgn][blk[i].ind] = i;
            }
        }
    }
    for(int i = 0; i < n; i++) {
        sufArr[pos[lgn-1][i]] = i;
    }
    for(int i = 1; i < n; i++) {
        lcp[i-1] = LCP(sufArr[i-1], sufArr[i]);
    }
}

```

1.5 Manacher's Algorithm

```

//Odd Palindromes
vector<int> d1(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i);
    while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
        k++;
    }
    d1[i] = k--;
}

```

```

    if (i + k > r) {
        l = i - k;
        r = i + k;
    }
}
//Even Palindromes
vector<int> d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
        k++;
    }
    d2[i] = k--;
    if (i + k > r) {
        l = i - k - 1;
        r = i + k;
    }
}

```

1.6 Palindrome Tree

```

const int MAXN = 105000;
struct node {
    int next[26];
    int len;
    int sufflink;
    int num;
};
int len;
char s[MAXN];
node tree[MAXN];
int num; // node 1 - root with len -1,
          // node 2 - root with len 0
int suff; // max suffix palindrome
long long ans;
bool addLetter(int pos) {
    int cur = suff, curlen = 0;
    int let = s[pos] - 'a';
    while (true) {
        curlen = tree[cur].len;
        if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos])
            break;
    }
}

```

```

        cur = tree[cur].sufflink;
    }
    if (tree[cur].next[let]) {
        suff = tree[cur].next[let];
        return false;
    }
    num++;
    suff = num;
    tree[num].len = tree[cur].len + 2;
    tree[cur].next[let] = num;
    if (tree[num].len == 1) {
        tree[num].sufflink = 2;
        tree[num].num = 1;
        return true;
    }
    while (true) {
        cur = tree[cur].sufflink;
        curlen = tree[cur].len;
        if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
            tree[num].sufflink = tree[cur].next[let];
            break;
        }
    }
    tree[num].num = 1 + tree[tree[num].sufflink].num;
    return true;
}

void initTree() {
    num = 2; suff = 2;
    tree[1].len = -1; tree[1].sufflink = 1;
    tree[2].len = 0; tree[2].sufflink = 1;
}

// gets(s);
// len = strlen(s);
// initTree();
// for (int i = 0; i < len; i++) {
//     addLetter(i);
// }

```

2 Combinatorial optimization

2.1 Dinic

```

#define ll long long int
#define inf ((ll)(1e18))

struct edge
{
    int a,b;
    ll cap,flow;
};

int n,src,dest,d[100005],ptr[100005];
vector<vector<int>> > v;
vector<edge> e;
// 100005 is the number of vertices
bool bfs()
{
    int i;
    queue<int> q;
    for(int i=0 ; i<n+2 ; i++) d[i]=-1;
    q.push(src);
    d[src]=0;
    while(!q.empty() && d[dest]==-1) {
        int u=q.front();
        q.pop();
        for(int i=0 ; i<v[u].size() ; i++) {
            int ind=v[u][i],vl=e[ind].b;
            ll cap=e[ind].cap,fl=e[ind].flow;
            if(d[vl]==-1 && fl<cap) {
                q.push(vl);
                d[vl]=d[u]+1;
            }
        }
    }
    return (d[dest]!=-1);
}

ll dfs(int u,ll curr)
{
    if(!curr) return 0;
    if(u==dest) return curr;
    for(;ptr[u]<v[u].size();++ptr[u]) {
        int ind=v[u][ptr[u]],vl=e[ind].b;
        if(d[vl]==d[u]+1 && e[ind].flow<e[ind].cap) {
            ll pushed=dfs(vl,min(curr,e[ind].cap-e[ind].flow));
            if(pushed>0) {

```

```

        e[ind].flow+=pushed;
        e[(ind^1)].flow-=pushed;
        return pushed;
    }
}
return 0;
}
ll dinic()
{
    ll flow=0,i;
    while(bfs()){
        for(int i=0 ; i<n+2 ; i++) ptr[i]=0;
        while(ll curr=dfs(src,inf))
            flow+=curr;
    }
    return flow;
}
void addedge(int x,int y,ll cap)
{
    edge e1={x,y,cap,0};
    edge e2={y,x,0,0};
    v[x].push_back((int)e.size());
    e.push_back(e1);
    v[y].push_back((int)e.size());
    e.push_back(e2);
}
//n=6 ;
//src=0 ; dest=3 ;
//v.resize(n+5) ;
//addedge(0,4,8) ;

// MIN-CUT BEGINS
const int N=1000 ;
vector<bool> vis(N,false) ;
vector<int> cut ;
void dfs(int node)
{
    if(vis[node])
        return ;
    cut.push_back(node) ;
    vis[node]=true ;
    for(int i=0; i<v[node].size() ; i++){
        int id=v[node][i] ;
        if(e[id].cap-e[id].flow<=0)
            continue ;
        int next=e[id].b ;
        dfs(next) ;
    }
}

```

```

}
// to get the vertices on the side of s in the min
// cut, first run max flow, then run dfs(source).
// The ans gets stored in the vector called cut.
// then just find which edges lead from some
// vertex in cut to some vertex not in cut.

```

2.2 Max bipartite matching

```

vector<bool> vis; // internal function for canMarry
// for right vertices
vector<int> man; // matching decided till now in
// terms of right vertices
vector<vector<int>> G; // graph from left->right
// vertices
int N, M; // N - left, M - right

int canMarry(int u) {
    for (auto v : G[u]) {
        if (vis[v])
            continue;
        vis[v]=true;
        if (man[v] == -1 || canMarry(man[v])) {
            man[v]=u;
            return 1;
        }
    }
    return 0;
}

int maxMtaching() {
    int ans=0;
    for (int i=0; i<N; ++i) {
        vis.assign(N, false); // very important
        // before every augmentation
        // augmentation step
        ans += canMarry(i);
    }
    return ans;
}

// G.resize(N); // left
// man.assign(M,-1); //right

```

2.3 Min Cost Max Flow

```

// as fast as dinic

```

```

// IMP mxN is the number of nodes in graph
const int mxN = 5500;
const int inf = 2e9;

struct Edgee {
    int to, cost, cap, flow, backEdge;
};

struct MCMF {
    int s, t, n;
    vector<Edgee> g[mxN];
    MCMF(int _s, int _t, int _n) {
        s = _s, t = _t, n = _n;
    }
    void addEdge(int u, int v, int cost, int cap) {
        Edgee e1 = { v, cost, cap, 0, g[v].size() };
        Edgee e2 = { u, -cost, 0, 0, g[u].size() };
        g[u].push_back(e1); g[v].push_back(e2);
    }
    pair<int, int> minCostMaxFlow() {
        int flow = 0, cost = 0;
        vector<int> state(n), from(n), from_edge(n), d(n);
        deque<int> q;
        while (true) {
            for (int i = 0; i < n; i++)
                state[i] = 2, d[i] = inf, from[i] = -1;
            state[s] = 1; q.clear(); q.push_back(s); d[s] = 0;
            while (!q.empty()) {
                int v = q.front(); q.pop_front(); state[v] = 0;
                for (int i = 0; i < (int) g[v].size(); i++) {
                    Edgee e = g[v][i];
                    if (e.flow >= e.cap || d[e.to] <= d[v] + e.cost)
                        continue;
                    int to = e.to; d[to] = d[v] + e.cost;
                    from[to] = v; from_edge[to] = i;
                    if (state[to] == 1) continue;
                    if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
                        q.push_front(to);
                    else q.push_back(to);
                    state[to] = 1;
                }
            }
            if (d[t] == inf) break;
            int it = t, addflow = inf;

```

```

            while (it != s) {
                addflow = min(addflow,
                    g[from[it]][from_edge[it]].cap
                    - g[from[it]][from_edge[it]].flow);
                it = from[it];
            }
            it = t;
            while (it != s) {
                g[from[it]][from_edge[it]].flow += addflow;
                g[it][g[from[it]][from_edge[it]].backEdge].flow -= addflow;
                cost += g[from[it]][from_edge[it]].cost * addflow;
                it = from[it];
            }
            flow += addflow;
        }
        return {cost, flow};
    }
};

//MCMF g(0,2*n+1,2*n+5) ; (source, target, total nodes)
//g.addEdge(i,j+n,cost,inf) ; (origin, target, cost, flow)
//g.minCostMaxFlow() ->returns a pair of {cost, flow}

```

2.4 Hopcroft Karp

```

#define MAX 100001
#define NIL 0
#define INF (1<<28)

// set the values of m and n
vector< int > G[MAX]; // only need to define edges from left to right
int n, m, match[MAX], dist[MAX];
// n: number of nodes on left side, nodes are numbered 1 to n
// m: number of nodes on right side, nodes are numbered n+1 to n+m
// G = NIL[0] 1 G1[G[1---n]] 1 G2[G[n+1---n+m]]

bool bfs() {
    int i, u, v, len;
    queue< int > Q;
    for(i=1; i<=n; i++) {

```

```

    if(match[i]==NIL) {
        dist[i] = 0;
        Q.push(i);
    }
    else dist[i] = INF;
}
dist[NIL] = INF;
while(!Q.empty()) {
    u = Q.front(); Q.pop();
    if(u!=NIL) {
        len = G[u].size();
        for(i=0; i<len; i++) {
            v = G[u][i];
            if(dist[match[v]]==INF) {
                dist[match[v]] = dist[u] + 1;
                Q.push(match[v]);
            }
        }
    }
}
return (dist[NIL]!=INF);
}

bool dfs(int u) {
    int i, v, len;
    if(u!=NIL) {
        len = G[u].size();
        for(i=0; i<len; i++) {
            v = G[u][i];
            if(dist[match[v]]==dist[u]+1) {
                if(dfs(match[v])) {
                    match[v] = u;
                    match[u] = v;
                    return true;
                }
            }
        }
        dist[u] = INF;
        return false;
    }
    return true;
}

int hopcroft_karp() {
    int matching = 0, i;
    // match[] is assumed NIL for all vertex in G
    while(bfs())
        for(i=1; i<=n; i++)
            if(match[i]==NIL && dfs(i))
                matching++;
    return matching;
}

```

```

}
//n=3 ; m=3 ;
//G[u].push_back(n+v) ;
//ans=hopcroft_karp()
// set match[i]=NIL if you run multiple times
// after running the algo match[i] stores what
// vertex it is matched to for both left and right
// vertices

```

3 Geometry

3.1 2D Geometry

```

typedef long double ld;
#define EPS 1e-9
const ld PI = 2 * acos((ld) 0);
#define T long double
struct PT {
    T x, y;
    PT() {}
    PT(T x, T y) : x(x), y(y) {}
    PT(const PT& p) : x(p.x), y(p.y) {}
    bool operator < (const PT& rhs) const {
        return make_pair(y, x) < make_pair(rhs.
            y, rhs.x);
    }
    bool operator == (const PT& rhs) const {
        return make_pair(y, x) == make_pair(rhs.
            y, rhs.x);
    }
    PT operator + (const PT& p) const {return
        PT(x + p.x, y + p.y);}
    PT operator - (const PT& p) const {return
        PT(x - p.x, y - p.y);}
    PT operator * (T c) const {return PT(x * c
        , y * c);}
    PT operator / (T c) const {return PT(x / c
        , y / c);}
};

T cross(PT p, PT q) {return p.x * q.y - p.y * q.x
    ;}
T area(PT a, PT b, PT c) {return fabs(cross(a, b)
    + cross(b, c) + cross(c, a)) / 2;}
T area2(PT a, PT b, PT c) {return cross(a, b) +
    cross(b, c) + cross(c, a);}
T dot(PT p, PT q) {return p.x * q.x + p.y * q.y;}
T dist(PT p, PT q) {return sqrt(dot(p - q, p - q))
    ;}

```

```

T dist2(PT p, PT q) {return dot(p - q, p - q);}
#define REMOVE_REDUNDANT
#ifdef REMOVE_REDUNDANT
bool between(const PT& a, const PT& b, const PT& c
) {
    return (fabs(area2(a, b, c)) < EPS && (a.x
        - b.x) * (c.x - b.x) <= 0 && (a.y - b.
            y) * (c.y - b.y) <= 0);
}
#endif
void ConvexHull(vector<PT>& pts) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()),
        pts.end());
    vector<PT> up, dn;
    for (int i = 0; i < pts.size(); i++) {
        while (up.size() > 1 && area2(up[
            up.size() - 2], up.back(), pts[
                i]) >= 0) up.pop_back();
        while (dn.size() > 1 && area2(dn[
            dn.size() - 2], dn.back(), pts[
                i]) <= 0) dn.pop_back();
        up.push_back(pts[i]);
        dn.push_back(pts[i]);
    }
    pts = dn;
    for (int i = up.size() - 2; i >= 1; i--)
        pts.push_back(up[i]);
#ifdef REMOVE_REDUNDANT
    if (pts.size() <= 2) return;
    dn.clear();
    dn.push_back(pts[0]);
    dn.push_back(pts[1]);
    for (int i = 2; i < pts.size(); i++) {
        if (between(dn[dn.size() - 2], dn[
            dn.size() - 1], pts[i])) dn.
            pop_back();
        dn.push_back(pts[i]);
    }
    if (dn.size() >= 3 && between(dn.back(),
        dn[0], dn[1])) {
        dn[0] = dn.back();
        dn.pop_back();
    }
    pts = dn;
#endif
}
PT RotateCCW90(PT p) {return PT(-p.y, p.x);}

```

```

PT RotateCW90(PT p) {return PT(p.y, -p.x);}
PT RotateCCW(PT p, T t) {return PT(p.x * cos(t) -
    p.y * sin(t), p.x * sin(t) + p.y * cos(t));}
PT ProjectPointLine(PT a, PT b, PT c) {
    return a + (b - a) * dot(c - a, b - a) /
        dot(b - a, b - a);
}
PT ProjectPointSegment(PT a, PT b, PT c) {
    T r = dot(b - a, b - a);
    if (fabs(r) < EPS) return a;
    r = dot(c - a, b - a) / r;
    if (r < 0) return a;
    if (r > 1) return b;
    return a + (b - a) * r;
}
T DistancePointSegment(PT a, PT b, PT c) {
    return dist(c, ProjectPointSegment(a, b, c
        ));
}
//Compute distance between PT (x, y, z) and plane
//ax + by + cz = d
T DistancePointPlane(T x, T y, T z, T a, T b, T c,
    T d) {
    return fabs(a * x + b * y + c * z - d) /
        sqrt(a * a + b * b + c * c);
}
//Determine if lines from a to b and c to d are
//parallel or collinear
int LinesParallel(PT a, PT b, PT c, PT d) {
    return fabs(cross(b - a, c - d)) < EPS;
}
int LinesCollinear(PT a, PT b, PT c, PT d) {
    return LinesParallel(a, b, c, d) && fabs(
        cross(a - b, a - c)) < EPS && fabs(
        cross(c - d, c - a)) < EPS;
}
//Determine if line segment from a to b intersects
//with line segment from c to d
int SegmentsIntersect(PT a, PT b, PT c, PT d) {
    if (LinesCollinear(a, b, c, d)) {
        if (dist2(a, c) < EPS || dist2(a,
            d) < EPS || dist2(b, c) < EPS
            || dist2(b, d) < EPS) return 1;
        if (dot(c - a, c - b) > 0 && dot(d
            - a, d - b) > 0 && dot(c - b,
            d - b) > 0) return 0;
        return 1;
    }
    if (cross(d - a, b - a) * cross(c - a, b -

```



```

        a) > 0) return 0;
    if (cross(a - c, d - c) * cross(b - c, d -
        c) > 0) return 0;
    return 1;
}
//Compute intersection of line passing through a
//and b
//with line passing through c and d, assuming that
//unique
//intersection exists; for segment intersection,
//check if
//segments intersect first
PT ComputeLineIntersection(PT a, PT b, PT c, PT d)
{
    b = b - a; d = c - d; c = c - a;
    return a + b * cross(c, d) / cross(b, d);
}
//Compute center of circle given three points
PT ComputeCircleCenter(PT a, PT b, PT c) {
    b = (a + b) / 2;
    c = (a + c) / 2;
    return ComputeLineIntersection(b, b +
        RotateCW90(a - b), c, c + RotateCW90(a
        - c));
}
//Determine if point is in a possibly non-convex
//polygon
//returns 1 for strictly interior points, 0 for
//strictly exterior points, and 0 or 1 for the
//remaining points.
int PointInPolygonSlow(const vector<PT>& p, PT q)
{
    int c = 0;
    for (int i = 0; i < p.size(); i++) {
        int j = (i + 1) % p.size();
        if ((p[i].y <= q.y && q.y < p[j].y
            || p[j].y <= q.y && q.y < p[i
                ].y) && q.x < p[i].x + (p[j].x
                - p[i].x) * (q.y - p[i].y) / (p
                [j].y - p[i].y)) c = !c;
    }
    return c;
}
//Strictly inside convex Polygon
#define Det(a, b, c) ((b.x - a.x) * (c.y - a.y) -
    (b.y - a.y) * (c.x - a.x))
int PointInPolygon(vector<PT>& p, PT q) {
    int a = 1, b = p.size() - 1, c;
    if (Det(p[0], p[a], p[b]) > 0) swap(a, b);

```

```

//Allow on edge --> if (Det... > 0 || Det
... < 0)
if (Det(p[0], p[a], q) > 0 || Det(p[0], p[
    b], q) < 0) return 0;
while (abs(a - b) > 1) {
    c = (a + b) / 2;
    if (Det(p[0], p[c], q) > 0) b = c;
    else a = c;
}
//Allow on edge --> return Det... <= 0
return Det(p[a], p[b], q) <= 0;
}
//Determine if point is on the boundary of a
//polygon
int PointOnPolygon(const vector<PT>& p, PT q) {
    for (int i = 0; i < p.size(); i++) if (
        dist2(ProjectPointSegment(p[i], p[(i +
            1) % p.size()], q), q) < EPS) return 1;
    return 0;
}
// Points should be in CCW or CW
T ComputeSignedArea(const vector<PT> &p) {
    T area = 0;
    for(int i = 0; i < p.size(); i++) {
        int j = (i+1) % p.size();
        area += p[i].x*p[j].y - p[j].x*p[i].y;
    }
    return area / 2.0;
}
// compute intersection of line through points a
//and b with
// circle centered at c with radius r > 0
vector<PT> CircleLineIntersection(PT a, PT b, PT c
    , T r) {
    vector<PT> ret;
    b = b-a;
    a = a-c;
    T A = dot(b, b);
    T B = dot(a, b);
    T C = dot(a, a) - r*r;
    T D = B*B - A*C;
    if (D < -EPS) return ret;
    ret.push_back(c+a+b*(-B+sqrt(D+EPS))/A);
    if (D > EPS)
        ret.push_back(c+a+b*(-B-sqrt(D))/A);
    return ret;
}
// compute intersection of circle centered at a

```

```

    with radius r
    // with circle centered at b with radius R
    vector<PT> CircleCircleIntersection (PT a, PT b, T
    r, T R) {
        vector<PT> ret;
        T d = sqrt(dist2(a, b));
        if (d > r+R || d+min(r, R) < max(r, R))
            return ret;
        T x = (d*d-R*R+r*r) / (2*d);
        T y = sqrt(r*r-x*x);
        PT v = (b-a)/d;
        ret.push_back(a+v*x + RotateCCW90(v)*y);
        if (y > 0)
            ret.push_back(a+v*x - RotateCCW90(
                v)*y);
        return ret;
    }

    // tests whether or not a given polygon (in CW or
    // CCW order) is simple
    bool IsSimple(const vector<PT> &p) {
        for (int i = 0; i < p.size(); i++) {
            for (int k = i+1; k < p.size(); k++) {
                int j = (i+1) % p.size();
                int l = (k+1) % p.size();
                if (i == l || j == k) continue;
                if (SegmentsIntersect(p[i], p[j],
                    p[k], p[l]))
                    return false;
            }
        }
        return true;
    }

```

4 Numerical algorithms

4.1 Number theory

```

    // returns g = gcd(a, b); finds x, y such that d =
    // ax + by
    int extended_euclid(int a, int b, int &x, int &y)
    {
        int xx = y = 0;
        int yy = x = 1;
        while (b) {
            int q = a / b;
            int t = b; b = a%b; a = t;

```

```

            t = xx; xx = x - q*xx; x = t;
            t = yy; yy = y - q*yy; y = t;
        }
        return a;
    }

    // computes b such that ab = 1 (mod n), returns -1
    // on failure
    int mod_inverse(int a, int n) {
        int x, y;
        int g = extended_euclid(a, n, x, y);
        if (g > 1) return -1;
        return mod(x, n);
    }

```

4.2 Systems of linear equations, Determinant

```

    int gauss (vector < vector<double> > &a, vector<
    double> &ans) {
        int n = (int) a.size();
        int m = (int) a[0].size() - 1;
        vector<int> where (m, -1);
        for (int col=0, row=0; col<m && row<n; ++col)
        {
            int sel = row;
            for (int i=row; i<n; ++i)
                if (abs (a[i][col]) > abs (a[sel][col]
                    ))
                    sel = i;
            if (abs (a[sel][col]) < EPS)
                continue;
            for (int i=col; i<=m; ++i)
                swap (a[sel][i], a[row][i]);
            where[col] = row;
            for (int i=0; i<n; ++i)
                if (i != row) {
                    double c = a[i][col] / a[row][col]
                        ];
                    for (int j=col; j<=m; ++j)
                        a[i][j] -= a[row][j] * c;
                }
            ++row;
        }
        ans.assign (m, 0);
        for (int i=0; i<m; ++i)
            if (where[i] != -1)
                ans[i] = a[where[i]][m] / a[where[i]][

```

```

        i];
for (int i=0; i<n; ++i) {
    double sum = 0;
    for (int j=0; j<m; ++j)
        sum += ans[j] * a[i][j];
    if (abs (sum - a[i][m]) > EPS)
        return 0;
}

for (int i=0; i<m; ++i)
    if (where[i] == -1)
        return INF;
return 1;
}

// Determinant of A = product of diagonal elements
// after applying gauss jordan method

```

4.3 Fast Fourier transform

```

using cd = complex<double>;
const double PI = acos(-1);
void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
}

```

```

    if (invert) {
        for (cd & x : a)
            x /= n;
    }

// a and b can be of arbitrary lengths not
// necessarily power of 2 here
// can easily tweak the code to multiply complex
// numbers too
vector<int> multiply(vector<int> const& a, vector<
int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin
(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

4.4 NTT

```

// If  $p=c \cdot 2^k+1$ , then we need  $2^k$  th root of unity
// root= not generator, rather  $g^c$ , root of unity
// , root_1 is its inverse wrt mod, root_pw means
//  $2^k$ 
// inverse(n,mod) is a function for modular
// inverse\
// for fft with arbitrary remainder, break
// coefficients into modulo sqrt(M), dont use CRT
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;
void fft(vector<int> & a, bool invert) {
    int n = a.size();

```

```

for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1)
        j ^= bit;
    j ^= bit;
    if (i < j)
        swap(a[i], a[j]);
}

for (int len = 2; len <= n; len <= 1) {
    int wlen = invert ? root_1 : root;
    for (int i = len; i < root_pw; i <= 1)
        wlen = (int)(1LL * wlen * wlen % mod);

    for (int i = 0; i < n; i += len) {
        int w = 1;
        for (int j = 0; j < len / 2; j++) {
            int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
            a[i+j] = u + v < mod ? u + v : u + v - mod;
            a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
            w = (int)(1LL * w * wlen % mod);
        }
    }

    if (invert) {
        int n_1 = inverse(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}

vector<int> multiply(vector<int> const& a, vector<
int> const& b) {
    vector<int> fa(a), fb(b);
    int n = 1;
    while (n < a.size() + b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = (fa[i] * 1LL * fb[i]) % mod;
    fft(fa, true);
    fa.resize(a.size() + b.size() - 1);
}

```

```

    return fa;
}

```

4.5 Online NTT

```

const long long int mod = 663224321;
const long long int root = 298744225;
const long long int rooti = 239546257;

int g[200005], w[(1<<18)], iw[(1<<18)], f[200005];
int a[200005], b[200005];

//modPow(a,b) = (a^b)%mod;
//void add(&a,b)
//void mul(&a,b)

void ntt(int *a, int n, bool invert){
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        for(int i = 0; i < n; i += len){
            int ind = 0, add = (1<<19)/len;
            for(int j = 0; j < len/2; j++){
                int u = a[i+j], v = (1LL*a[i+j+len/2] * (invert ? iw[ind] : w[ind]))%mod;
                a[i+j] = (u+v)%mod;
                a[i+j+len/2] = (u-v+mod)%mod;
                ind += add;
            }
        }
    }

    if (invert) {
        int invn = modPow(n, mod-2);
        for (int i = 0; i < n; i++) mul(a[i], invn);
    }
}

void multiply(int l1, int r1, int l2, int r2, int k){
    for(int i = l1; i <= r1; i++){
        a[i-l1] = f[i];
    }
}

```

```

for(int i = r1-l1+1; i < k; i++){
    a[i] = 0;
}
for(int i = l2; i <= r2; i++){
    b[i-l2] = g[i];
}
for(int i = r2-l2+1; i < k; i++){
    b[i] = 0;
}
ntt(a, k, false);
ntt(b, k, false);
for(int i = 0; i < k; i++){
    mul(a[i], b[i]);
}
ntt(a, k, true);
for(int i = 0; i < k; i++){
    if(l1 + l2 + 1 + i >= 100005) break;
    add(f[l1+l2+i+1], a[i]);
}
}

void precompute f() {
    f[0] = f[1] = f[2] = 1;
    for(int i = 1; i < 100005; i++){
        add(f[i+1], f[i]);
        add(f[i+2], f[i]);
        int k = 2;
        while(i%k == 0){
            multiply(i-k, i-1, k, 2*k-1, 2*k);
            k *= 2;
        }
    }
}

//w, iw - Powers of root and rooti respectively;
//Compute g - Array to be multiplied with
//precompute f

```

4.6 Karatsuba

```

void copy_vec(vector<int> &a, vector<int> &b, int
l, int r){
    b.clear();
    for(int i = l; i < r; i++)
        b.push_back(a[i]);
}

void karatsuba(vector<int> &a, vector<int> &b,
vector<int> &c, int n){
    c.resize(2*n-1, 0);

```

```

if(n <= 32){
    for(int i = 0; i < n; i++){
        if(a[i] == 0) continue;
        for(int j = 0; j < n; j++){
            c[i+j] += a[i]*b[j];
        }
    }
    return;
}

vector<int> a1, a2, b1, b2, c1, c2, c3;
copy_vec(a, a1, 0, n/2);
copy_vec(a, a2, n/2, n);
copy_vec(b, b1, 0, n/2);
copy_vec(b, b2, n/2, n);
karatsuba(a1, b1, c1, n/2);
karatsuba(a2, b2, c2, n/2);
for(int i = 0; i < n/2; i++){
    a1[i] += a2[i];
    b1[i] += b2[i];
}
karatsuba(a1, b1, c3, n/2);
for(int i = 0; i < n-1; i++){
    c[i] += c1[i];
    c[i+n] += c2[i];
    c[i+n/2] += c3[i] - c1[i] - c2[i];
}

void multiply(vector<int> &a, vector<int> &b,
vector<int> &c){
    int sz = 1, n = a.size(), m = b.size();
    while(sz <= max(n,m)) sz <= 1;
    while(a.size() < sz) a.push_back(0);
    while(b.size() < sz) b.push_back(0);
    karatsuba(a, b, c, sz);
}

```

5 Graph algorithms

5.1 Strongly connected components and Topological Sort

```

#include <bits/stdc++.h>
using namespace std;
// nodes from 0 to N-1
// dfs1- can be used for topological sort of DAG

```

```

int n;
vector<vector<int>> > g, gr;
vector<bool> used;
vector<int> order, component, scc;
vector<vector<int>> > comp;

void dfs1 (int v) {
    used[v] = true;
    for (size_t i=0; i<g[v].size(); ++i)
        if (!used[ g[v][i] ])
            dfs1 (g[v][i]);
    order.push_back (v);
}

void dfs2 (int v) {
    used[v] = true;
    component.push_back (v);
    for (size_t i=0; i<gr[v].size(); ++i)
        if (!used[ gr[v][i] ])
            dfs2 (gr[v][i]);
}

//scc - component no
//order- reverse of topological sort of the points
//      (can be modified for topological sort of SCC
//      components)
void scc_calc() {
    used.assign (n, false);
    for (int i=0; i<n; ++i)
        if (!used[i])
            dfs1 (i);
    used.assign (n, false);
    scc.resize(n);
    for (int i=0, cnt = 0; i<n; ++i) {
        int v = order[n-1-i];
        if (!used[v]) {
            dfs2 (v);
            for (int j=0 ; j<component.size() ; j
                ++){
                scc[component[j]]=cnt;
            }
            comp.push_back (component);
            cnt++;
            component.clear();
        }
    }
}

int main() {

```

5.2 Eulerian path

```

struct Edge;
typedef list<Edge>::iterator iter;

struct Edge
{
    int next_vertex;
    iter reverse_edge;

    Edge(int next_vertex)
        :next_vertex(next_vertex)
        { }
};

const int max_vertices=;
int num_vertices;
list<Edge> adj[max_vertices];           //
    adjacency list

vector<int> path;

//Stores the final path in 'path' vector
void find_path(int v)
{
    while (adj[v].size() > 0)
    {
        int vn = adj[v].front().
            next_vertex;
        adj[vn].erase(adj[v].front().
            reverse_edge);
        adj[v].pop_front();
        find_path(vn);
    }
    path.push_back(v);
}

void add_edge(int a, int b)
{
    adj[a].push_front (Edge(b));
    iter ita = adj[a].begin();
    adj[b].push_front (Edge(a));
    iter itb = adj[b].begin();
    ita->reverse_edge = itb;
    itb->reverse_edge = ita;
}

```

5.3 2 SAT

```
//2*k - normal vertex k; 2*k + 1 - negation vertex
// of k
// g - implications; gt - reverse of gt;
```

```
int n;
vector<vector<int>> g, gt;
vector<bool> used;
vector<int> order, comp;
vector<bool> assignment;

void dfs1(int v) {
    used[v] = true;
    for (int u : g[v]) {
        if (!used[u])
            dfs1(u);
    }
    order.push_back(v);
}

void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : gt[v]) {
        if (comp[u] == -1)
            dfs2(u, cl);
    }
}

bool solve_2SAT() {
    used.assign(n, false);
    for (int i = 0; i < n; ++i) {
        if (!used[i])
            dfs1(i);
    }

    comp.assign(n, -1);
    for (int i = 0, j = 0; i < n; ++i) {
        int v = order[n - i - 1];
        if (comp[v] == -1)
            dfs2(v, j++);
    }

    assignment.assign(n / 2, false);
    for (int i = 0; i < n; i += 2) {
        if (comp[i] == comp[i + 1])
            return false;
        assignment[i / 2] = comp[i] > comp[i + 1];
    }
    return true;
}
```

5.4 SPFA

```
const int MAXN = 100005;
const long long int inf = (1LL<<62);
vector<pair<int,int> > g[MAXN];
int n, cnt[MAXN];
long long int dist[MAXN];
bool inQueue[MAXN];

bool spfa(int s) {
    for (int i = 1; i <= n; i++) {
        dist[i] = inf;
        cnt[i] = 0;
        inQueue[i] = false;
    }
    queue<int> myQ;
    myQ.push(s);
    dist[s] = 0;
    inQueue[s] = true;
    cnt[s]++;
    while (!myQ.empty()) {
        int v = myQ.front();
        myQ.pop();
        inQueue[v] = false;
        for (int i = 0; i < g[v].size(); i++) {
            if (dist[g[v][i].first] > dist[v] + g[v][i].second) {
                dist[g[v][i].first] = dist[v] + g[v][i].second;
                if (!inQueue[g[v][i].first]) {
                    inQueue[g[v][i].first] = true;
                    myQ.push(g[v][i].first);
                    cnt[g[v][i].first]++;
                    if (cnt[g[v][i].first] >= n) {
                        return true;
                    }
                }
            }
        }
    }
    return true;
}
```

5.5 Bridge Tree

```
#define VI vector<int>
```

```

const int N=1e5+5;
const int M=1e5+5;
VI tree[N], graph[N], wt[N], aj[N];
int U[M], V[M], cost[M];
bool isbridge[M];
int visited[N], arr[N], cmp[N], T=0;
int head[N];
int cmpno=0;
queue<int> Q[N];
int adj(int u, int e) {
    return U[e]==u?V[e]:U[e];
}

int dfs0(int u, int edge) {
    visited[u]=1;
    arr[u]=T++;
    int dbe=arr[u];
    for(auto e:graph[u]) {
        int w=adj(u, e); // w-> adjacent
        node
        if(!visited[w])
            dbe=min(dbe, dfs0(w, e));
        else if(e!=edge)
            dbe=min(dbe, arr[w]);
    }
    if(dbe==arr[u] && edge!=-1)
        isbridge[edge]=true;
    return dbe;
}

void dfs1(int v) {
    int currcmp=cmpno;
    Q[currcmp].push(v);
    visited[v]=1;
    head[cmpno]=v;
    cmp[v]=currcmp;
    while(!Q[currcmp].empty()) {
        int u=Q[currcmp].front();
        Q[currcmp].pop();
        for(auto e:graph[u]) {
            int w=adj(u, e); // w->
            adjacent node
            if(visited[w]) continue;
            if(isbridge[e]) {
                cmpno++;
                tree[currcmp].
                    push_back(cmpno);
                tree[cmpno].
                    push_back(
                        currcmp);
            }
        }
    }
}

```

```

    wt[currcmp].
        push_back(cost[
            e]);
    wt[cmpno].
        push_back(cost[
            e]);
    dfs1(w);
}
else{
    Q[currcmp].push(w);
    cmp[w]=currcmp;
    visited[w]=1;
}
}
}

// graph[node].push_back(edge_num) GRAPH stores
// corresponding edge numbers for each node
// U[edge_num]=node1, V[edge_num]=node2, cost[
// edge_num]-> if it has cost(in that case bridge
// tree will return in wt[], cost also of tree)
// can create another vector adjacent[u] for
// storing adjacent vertices
//set visited[N] to 0, T=0, call dfs0(v, -1)->
// marks bridge edges
// call dfs1(v), forms bridge tree in "tree" ->
// nodes are component nos., starting from 0 to
// cmpno incl.
// cmp[i]-> comp no of ith node in bridge tree,
// head[cmpno]->head node of component cmpno

```

5.6 Heavy Light Decomposition

```

// g - graph, head[v] - head of chain to which v
// belongs to
//heavy[v] - child with max size, tree - base
// array of seg tree

const int MAXN = 100005;
const int LGN = 18;

vector<pair<int, int> > g[MAXN];
int head[MAXN], heavy[MAXN], pos[MAXN], parent[LGN]
    [MAXN], depth[MAXN], base[MAXN], weight[MAXN];
int ptr, n;
int tree[4*MAXN];

int dfs(int v, int p) {
    int size = 1, max_size = 0;

```



```

parent[0][v] = p;
for(int i = 0; i < g[v].size(); i++){
    int u = g[v][i].first;
    if(u != p){
        depth[u] = depth[v] + 1;
        weight[u] = g[v][i].second;
        int tmp = dfs(u, v);
        size += tmp;
        if(max_size < tmp) heavy[v] = u;
    }
}
return size;
}

void hld(int v, int h){
    head[v] = h;
    base[ptr] = v;
    pos[v] = ptr++;
    if(heavy[v] != -1){
        hld(heavy[v], h);
    }
    for(int i = 0; i < g[v].size(); i++){
        int u = g[v][i].first;
        if(u != parent[0][v] && u != heavy[v]){
            hld(u, u);
        }
    }
}

int lca(int u, int v){
    if(depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for(int i = 0; i < LGN; i++){
        if(diff & (1 << i)) u = parent[i][u];
    }
    if(u == v) return u;
    for(int i = LGN-1; i >= 0; i--){
        if(parent[i][u] != parent[i][v]){
            u = parent[i][u]; v = parent[i][v];
        }
    }
    return parent[0][u];
}

void buildTree(int v, int l, int r){
    if(l == r){
        tree[v] = weight[base[l]];
        return;
    }
    int m = (l+r)/2;
    buildTree(2*v, l, m);

```

```

    buildTree(2*v+1, m+1, r);
    tree[v] = tree[2*v] + tree[2*v+1];
}

void update(int v, int l, int r, int ind, int val){
    if(l == r){
        tree[v] = val;
        return;
    }
    int m = (l+r)/2;
    if(ind <= m) update(2*v, l, m, ind, val);
    else update(2*v+1, m+1, r, ind, val);
    tree[v] = tree[2*v] + tree[2*v+1];
}

int query(int v, int s, int e, int l, int r){
    if(l <= s && e <= r) return tree[v];
    if(e < l || r < s) return 0;
    int m = (s+e)/2;
    return query(2*v, s, m, l, r) + query(2*v+1, m
        +1, e, l, r);
}

int queryAncestorPath(int u, int v){
    int ans = 0;
    while(true){
        if(head[u] == head[v]){
            if(u == v) break;
            ans += query(1, 1, n, pos[v] + 1, pos[
                u]);
            break;
        }
        ans += query(1, 1, n, pos[head[u]], pos[u
            ]);
        u = parent[0][head[u]];
    }
    return ans;
}

int queryPath(int u, int v){
    int l = lca(u, v);
    return queryAncestorPath(u, l) +
        queryAncestorPath(v, l);
}

// memset(heavy, -1, sizeof(heavy));
// dfs(1, 0);
// hld(1, 1);
// buildTree(1, 1, n);
// for(int k = 1; k < LGN; k++){
//     for(int i = 1; i <= n; i++){

```

```
//      parent[k][i] = parent[k-1][parent[k-1][
// i]];
// }
// }
```

5.7 Centroid Decomposition

```
// n - total number of nodes in tree
// compute - function to be computed

const int MAXN = 100005;
vector<int> g[MAXN];
int dead[MAXN], sz[MAXN], parent[MAXN];

void dfs(int v, int p) {
    sz[v] = 1;
    for(int i = 0; i < g[v].size(); i++) {
        int u = g[v][i];
        if(u != p) {
            dfs(u, v);
            sz[v] += sz[u];
        }
    }
}

void cd(int v, int cnt, int p) {
    int bg = -1;
    for(int i = 0; i < g[v].size(); i++) {
        if(dead[g[v][i]] == 0) {
            if(bg == -1 || sz[bg] < sz[g[v][i]]) {
                bg = g[v][i];
            }
        }
    }
    if(bg == -1) {
        ans[v] = ch;
    } else if(sz[bg] <= cnt/2) {
        parent[v] = p;
        compute(v);
        dead[v] = 1;
        for(int i = 0; i < g[v].size(); i++) {
            if(dead[g[v][i]] == 0) {
                cd(g[v][i], sz[g[v][i]], v);
            }
        }
    } else {
        sz[v] = cnt - sz[bg];
        sz[bg] = cnt;
    }
}
```

```
cd(bg, cnt, p);
}

// dfs(1, 0);
// memset(dead, 0, sizeof(dead));
// cd(1, n, 0);
```

6 Data Structures

6.1 Treap

```
#include <bits/stdc++.h>
using namespace std;

const int NMAX=300010;
int N=0;
vector<char> ans;
int posl, posr;

struct node {
    int l, r, cnt; // cnt->number of nodes in
    subtree
    char val;
    long long pr;
    int lazy;
};

node nodes[NMAX];

inline void upd(int x) {
    if(x==-1)
        return;
    int l, r;
    l = nodes[x].l; r = nodes[x].r;
    nodes[x].cnt=1;
    if(l >= 0) nodes[x].cnt += nodes[l].cnt;
    if(r >= 0) nodes[x].cnt += nodes[r].cnt;
}

inline void lazy(int x) {
    if(x==-1)
        return;
    if(nodes[x].lazy==0) {
        return;
    }
    int l, r;
    l=nodes[x].l; r=nodes[x].r;
    nodes[x].l=r; nodes[x].r=l;
    if(l!=-1)
```

```

        nodes[l].lazy^=1 ;
    if(r!=-1)
        nodes[r].lazy^=1 ;
    nodes[x].lazy=0 ;
    return;
}

int create_node(char val){
    nodes[N].l = nodes[N].r = -1;
    nodes[N].val = val ;
    nodes[N].cnt = 1;
    nodes[N].pr = rand();
    nodes[N].lazy=0 ;
    return N++;
}

inline int count(int node)
{
    return (node==-1)? 0:nodes[node].cnt ;
}

pair<int,int> split(int root, int pos, int add){
    if(root < 0) return make_pair(-1,-1);

    pair<int,int> sp;
    //lessequal greater
    lazy(root) ;
    int cur=add+1+count(nodes[root].l) ;
    if(pos < cur){
        sp = split(nodes[root].l,pos,add);
        nodes[root].l = sp.second;
        sp.second = root;
        upd(root);
        return sp;
    }

    sp = split(nodes[root].r,pos,cur);
    nodes[root].r = sp.first;
    sp.first = root;
    upd(root);
    return sp;
}

int merge(int l, int r){
    if(l < 0 && r < 0) return -1;
    lazy(l) ; lazy(r) ;
    if((l >= 0) && (r < 0 || nodes[l].pr >
        nodes[r].pr)){
        nodes[l].r = merge(nodes[l].r,r);
        upd(l);
        return l;
    }

```

```

    }
    nodes[r].l = merge(l,nodes[r].l);
    upd(r);
    return r;
}

int rt=-1 ;
void ins(char val){ // only valid for left->right
    order of insertion
    int n = create_node(val);
    rt= merge(rt,n);
}

```

6.2 Persistent Segment Tree

```

const int MAXN = 100005;
const int LN = 20;

int n, queries;
int tree[MAXN*LN][2], val[MAXN*LN], root[MAXN];
int ptr = 1;

int update(int v, int l, int r, int index, int x){
    if(l == r){
        val[ptr] = min(val[v], x);
        tree[ptr][0] = 0;
        tree[ptr][1] = 0;
        ptr++;
        return ptr-1;
    }
    int mid = (l+r)/2;
    int nv = ptr++;
    tree[nv][0] = tree[v][0];
    tree[nv][1] = tree[v][1];
    if(index <= mid) tree[nv][0] = update(tree[v][0], l, mid, index, x);
    else tree[nv][1] = update(tree[v][1], mid+1, r, index, x);
    val[nv] = max(val[tree[nv][0]], val[tree[nv][1]]);
    return nv;
}

int main(){
    tree[0][0] = tree[0][1] = 0;
    val[0] = 1e9 + 5;
    root[0] = 0;
    for(int i = 1; i <= queries; i++){
        root[i] = update(root[i-1], 1, n, index, x);
    }
}

```

} }

7 Important Formulae

7.1 Partition Numbers

$$\sum_{i=0}^n \alpha(i)p(n-i) = 0, n \geq 1,$$

$$\text{where } p(0) = 1 \text{ and } \alpha(n) = \begin{cases} 1, & \text{if } n = 0 \\ (-1)^r & \text{if } n = \frac{r(3r \pm 1)}{2}, r \geq 1 \\ 0 & \text{otherwise} \end{cases}$$

7.2 Transformation matrix for XOR FFT

$$H_{2^n} = \frac{1}{\sqrt{2}} \begin{bmatrix} +H_{2^{n-1}} & +H_{2^{n-1}} \\ +H_{2^{n-1}} & -H_{2^{n-1}} \end{bmatrix}$$

$$H_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$H^{-1} = H$$

7.3 Transformation matrix for AND FFT

$$H_{2^n} = \begin{bmatrix} 0 & H_{2^{n-1}} \\ H_{2^{n-1}} & H_{2^{n-1}} \end{bmatrix}$$

$$H_2 = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

$$H_{2^n}^{-1} = \begin{bmatrix} -H_{2^{n-1}}^{-1} & H_{2^{n-1}}^{-1} \\ H_{2^{n-1}}^{-1} & 0 \end{bmatrix}$$

$$H_2^{-1} = \begin{bmatrix} -1 & 1 \\ 1 & 0 \end{bmatrix}$$

7.4 Transformation matrix for OR FFT

$$H_{2^n} = \begin{bmatrix} H_{2^{n-1}} & H_{2^{n-1}} \\ H_{2^{n-1}} & 0 \end{bmatrix}$$

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$H_{2^n}^{-1} = \begin{bmatrix} 0 & H_{2^{n-1}}^{-1} \\ H_{2^{n-1}}^{-1} & -H_{2^{n-1}}^{-1} \end{bmatrix}$$

$$H_2^{-1} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

8 Templates

8.1 Inbuilt functions and Data Structures

```
//Builtin Function
bool __builtin_add_overflow(type1 a, type2 b,
                             type3 *res)
bool __builtin_sub_overflow(type1 a, type2 b,
                             type3 *res)
bool __builtin_mul_overflow(type1 a, type2 b,
                             type3 *res)
int __builtin_popcount(int a);
int __builtin_popcountll(long long int a);

//Header files for PBDS
#include <ext/pb_ds/assoc_container.hpp> // Common
file
#include <ext/pb_ds/tree_policy.hpp> // Including
tree_order_statistics_node_update

//Namespace
using namespace __gnu_pbds; // New
using namespace pb_ds; // Old

//Tree template
template<
typename Key, // Key type
typename Mapped, // Mapped-policy
```

```

typename Cmp_Fn = std::less<Key>, // Key
        comparison functor
typename Tag = rb_tree_tag, // Specifies which
        underlying data structure to use
template<
typename Const_Node_Iterator,
typename Node_Iterator,
typename Cmp_Fn_,
typename Allocator_>
class Node_Update = null_node_update, // A policy
        for updating node invariants
typename Allocator = std::allocator<char> > // An
        allocator type
class tree;

```

```

//Ordered Set
typedef tree<
int,
null_type,
less<int>,
rb_tree_tag,
tree_order_statistics_node_update>
ordered_set;

//Member Functions
int find_by_order(int ind) //Returns element at
        position ind
int order_of_key(int val) //Returns number of
        items strictly smaller than val

```